Behavioral Cloning With A Raspberry Pi Car

Trevor Chartier
Department of Computer Science
Colorado State University
Fort Collins, United States
trevor.chartier@colostate.edu

Braden Hollis
Department of Computer Science
Colorado State University
Fort Collins, United States
bhollis6@colostate.edu

***Abstract*—This paper showcases an end-to-end behavioral cloning system for autonomous steering on a Raspberry Pi 5. We address control instability, distribution shift, and hardware latency. These are all major challenges in embedded imitation learning. Unlike traditional regression approaches, we framed this task as a classification problem. Using classification made the model more robust to noise and inverse-steering errors. To address distribution shift, we used Expert Intervention Learning (EIL). Furthermore, we used Pruning, Clustering, and Quantization Aware Training (PCQAT) pipeline for real-time inference on the CPU.**

Our experiments showed that our classification-based policy outperformed random policy baselines. A single iteration of EIL increased the average autonomous driving distance by 4.5 times compared to a random policy. PCQAT optimization reduced the model size by 9.4 times (from 665.7KB to 70.7KB) while yielding an 86.08 test accuracy. It showed that deep CNNs can be effectively deployed on embedded platforms without accelerators. Also, we used Grad-CAM saliency maps which showed that while the model can extract lane features, it still is susceptible to relying on the background environment. Lastly, we found that most data augmentation techniques, especially horizontal flipping, proved to degrade performance.

I. Introduction

This project implements an end-to-end steering control system for an autonomous scale car. A deep convolutional neural network (CNN) performs real-time steering controls using camera images. The model is deployed on a Raspberry Pi 5.

We are using behavioral cloning to imitate the actions of an expert. Our model is evaluated quantitatively (accuracy on correct steering class prediction and distance traveled autonomously) and qualitatively (video of autonomous driving). We will also be evaluating the memory footprint, and compression ratio of a pruned and quantized model over the original model.

We implemented EIL [1] on the embedded platform. Imitation learning often fails due to a distribution shift problem — data encountered at runtime often varies significantly from training data. EIL is an iterative process to address this issue. We compare the performance after various iterations of the EIL process.

We wish to evaluate the effectiveness of Behavior Cloning techniques applied to low level control of an autonomous, scale car in the real world. We will address the feasibility of deploying behavior cloning on a low power, embedded device across multiple real-world environments (two tracks) with varying complexities.



Fig. 1. Track 1 for the Raspberry Pi Car with a side-by-side diagram.

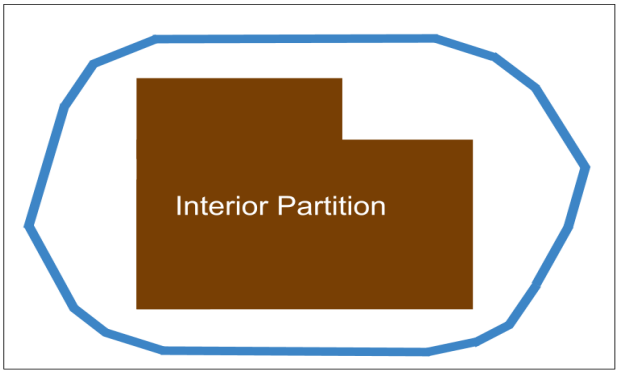


Fig. 2. Mockup of Track 2 for the Raspberry Pi Car

We use a modified hobby-grade RC car as the hardware platform for this project. Computing is handled by an onboard Raspberry Pi 5 with 4GB of SDRAM, 32 GB of microSD flash storage, and a 64-bit-quad-core processor. The Raspberry Pi sends steering and throttle commands over I2C bus to a PCA9685 servo driver. Based on the commands it receives, the servo driver generates and sends the appropriate pulse width modulation (PWM) signals to both the servo motor for steering control and the electronic speed controller (ESC) for throttle control. The Raspberry Pi is powered from the RC car's existing 8.4V Nickel Metal Hydride (NiMH) battery, stepped down to

5V with a DC-DC Buck converter. Finally, the Raspberry Pi receives images from a 5MP camera. A wiring overview of the system can be found in Figure 3.[1]

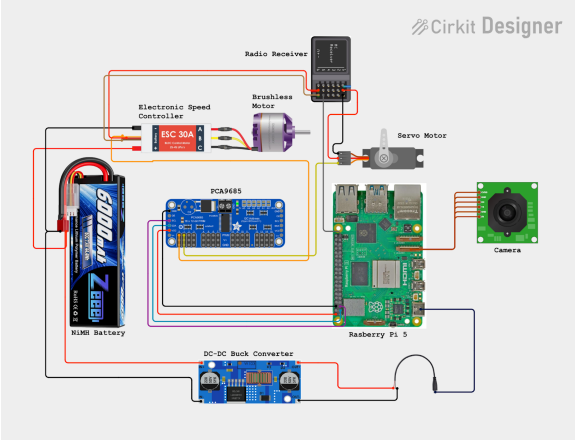


Fig. 3. Mockup Raspberry Pi Car.

II. Related Work

A. End-to-end Learning (PilotNet).

PilotNet [2] is a major work in the autonomous steering field by Bojarski et al. in 2016. The authors showed that a CNN could be used to map images from a front-facing camera to steering commands. It circumvents the need for lane markers or path planning algorithms. The authors used three cameras (right, center, and left) to collect data and used shifts and rotations to augment data.

Our project drew inspiration from PilotNet and their architecture. Our CNN was based on their model, and we tried using the YUV color space as outlined in the paper. However, we diverged from their approach by reframing the task as a classification problem and instead of using a three-camera setup to generate recovery data, we used EIL.

B. Distribution Shift

A major challenge in imitation learning is the "distribution shift" problem. In this problem, the agent encounters an unseen state. There are two methods to address this. Torabi et al. [3] uses Behavioral Cloning from Observation (BCO). BCO involves the agent only using unlabeled expert observations and learning the mapping between observed states and the actions the expert took. This is a great technique with large amounts of data, but we were collecting our own data.

Instead, we opted for Dataset Aggregation (DAgger) as proposed by Ross et al. [4]. DAgger requires ground-truth labels, but guarantees our model receives the correct input. Additionally, labeling was done automatically (see Section III).

C. Autonomous RC Platforms and Latency

Haji et al. [5] tried behavioral cloning on a Raspberry Pi 3. They demonstrated autonomous lane keeping but were bottlenecked by the hardware latency. They suggested GPU-accelerated hardware could solve their issue.

In contrast, we solved our latency constraints using software optimization. By applying PCQAT, our moderately deep CNN could run on a standard Raspberry Pi 5 without sacrificing accuracy. It solved the latency issues outlined in [5] without requiring a GPU upgrade.

D. Data Imbalance and Distribution

Our class imbalance results were not an anomaly. The dominance of a "straight" driving class is well-documented. Morga-Bonilla et al. [6] show strategies to address this. The authors use synthetic over-sampling and geometric augmentation. They also highlight that failure to fix this class imbalance in driving problems leads to poor performance.

Our approach also found that class balancing was an issue, and we used class weights to bias the left driving direction. However, we found that this unbalanced dataset proved to be a better representation of driving data than a perfectly balanced dataset—and in turn yielded better results.

III. Data

To collect our datasets, we created a python script that allows for teleoperation of the car's steering by a human expert through keyboard input. The human attempts to follow the tape line track closely. The control loop runs at approximately 57hz, bottlenecked by the camera frame capture rate. At each control timestep, the expert steering command is received and sent to the servo motors. Throttle is not controlled by the expert; instead, it is pulsed on and off at regular, predefined intervals. For example, throttle is set to the minimum speed for 15 control cycles and then set to 0 speed for the next 30 cycles. This pulsation is to make the car move more slowly than the motor's lowest speed otherwise would allow. During teleoperation, camera frames are captured and logged alongside the expert's steering command at that control timestep. Additional metadata is also logged. We experimented with different types of metadata. The image, expert command, and metadata make up one data sample.

We collected three separate datasets using a Raspberry Pi Camera at 640x480 pixel resolution. Each dataset is collected under different conditions to allow for experimentation.

1) 57,969 images on Track 1 with steering and throttle labels. Steering labels were floats and throttle labels were booleans. Data was captured at 57 frames per second. Throughout data collection on Track 1, light conditions were varied by toggling lights and opening and closing blinds. Additionally, we captured 8 shorter trials where the car started from far from the track and drove back to the track. This will help the car recover from difficult states.

[1] The hardware platform was adapted from a previous project, LAYMO [7]

2) 85,219 samples from Track 2. Samples are collected at 57hz. The dataset represents approximately 25 minutes of driving time. Unlike dataset 1, steering commands are discrete; they consist of full left, straight, or full right. Additional metadata is logged consisting of the last 2 seconds of steering data (taken at 0.1 second intervals), and the index within the throttle pulse cycle. Artificial lighting conditions were kept consistent; however because the dataset was collected at various times of day, natural light still varied.

3) 19,592 samples from Track 2. This dataset was collected under nearly identical conditions to daataset 2. The only difference was that samples were collected at only 5.7 hz. Therefore, this dataset, despite being smaller, represents a much larger drive time of approximately 57 minutes. This was in an effort to get more variation within the dataset.

Dataset 1 was used to train regression and classification models, described in Section IV. Dataset 2 was down sampled by 50% and combined with dataset 3 to train classification models. The classification models showed much more promising real-world performance and thus are used for the rest of this study.

A. Preprocessing

During preprocessing, each image is normalized and reduced from 640 by 480 pixels to 90 by 60 pixels.



Fig. 4. Six random 90x60-pixel images captured by the Pi Camera.

To prevent data leakage, we structured our training, testing, and validation datasets by trial (a single recording session) rather than by individual frames.

In our initial approach, we shuffled and randomly split our dataset into train, test, and validation sets. The implication of this approach is that training and testing data had a significant correlation between consecutive frames. This allowed for memorization and artificially inflated validation metrics. By isolating entire trials, the model was evaluated on truly unseen data, and the validation and test sets more accurately reflect the demands of our model during deployment.

B. Cleaning

While rare, we ran into instances of mislabeled data. To detect this, we converted each image to the HSV color space and then used OpenCV to create a mask and calculate the centroid of the blue tape. If the centroid was far left (marked by the pink vertical line) and the label was to turn right, the image was removed. This process was repeated vice versa for the right side using a specified threshold.



Fig. 5. An image passed through the data cleaning algorithm with a centroid at 2 pixels from the left and a steering label 1.0 (right).

We took a conservative approach to data cleaning by omitting images with straight steering labels and setting the threshold to the outer edges of the image.

IV. METHODS

A. Region of Interest Cropping and YUV Conversion

Generalization was one of our highest priorities when designing our pipeline. When running into generalization issues, we decided to crop the top 25 pixels of the input image to encourage the model to learn patterns related to the position of the blue tape line rather than focusing on features in the background.

Another technique that we tried was converting the images from RGB to YUV in the model. The YUV color space helps the blue line stand out even in vastly different lighting conditions. The NVIDIA PilotNet paper makes use of this approach as well. [2].

However, using these techniques yielded no significant benefit in performance metrics. Therefore, we left it out of our final model.

B. Data Augmentation

Data augmentation proved crucial as our first models proved unable to generalize to make good predictions at inference time, despite having strong performance on the validation and test sets. Although the validation and test sets come from separate trials, they are still collected at similar times of day as the training data. This may cause the model to overfit to particular lighting conditions. Augmentation was able to help with this issue; however, picking which transformations to use was not entirely straightforward. We experimented with many different data augmentation techniques: random brightness, Gaussian noise, random zoom, random translation, random rotation, and horizontal flipping. Augmentation techniques are applied to the training data. Validation and test data is not augmented. On average, incorporating all of these techniques produced a test set loss of 0.5861 and an accuracy of 0.7829.

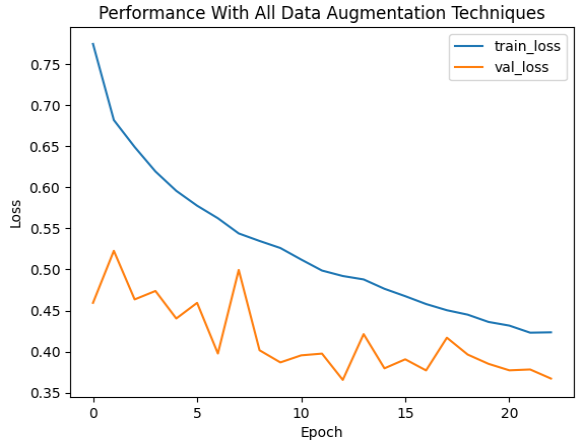


Fig. 6. Training and validation sparse categorical cross-entropy loss of all data augmentation techniques.

Horizontal flipping was a special case where we tried to use it as an up-sampling strategy (discussed in Section IV.C) to increase the number of images with steering labels of -1 or 1. Yet, it yielded poor performance. Ultimately, we decided only to use random brightness which increased performance dramatically to an average test set loss of 0.4903 and an accuracy of 0.8539.

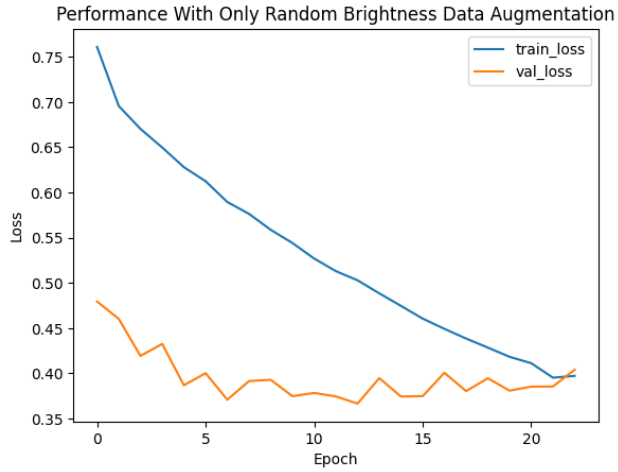


Fig. 7. Training and validation sparse categorical cross-entropy our final model (only random brightness data augmentation).

C. Class Balancing

A significant challenge in behavioral cloning is class imbalance. In our data and most driving data, straight was the dominant class. But this can lead to the model predicting straight as a "safe" class that minimizes loss.

To address this, we used three approaches:

1. We down sampled 50% of our original (mostly straight) data and concatenated it with 100% of an EIL iteration (see Section III.F). The original data favored straight labels and the EIL iterations—focused on recovery maneuvers—had mostly left and right labels.
2. Up-sampling left and right classes by using horizontal flipping. This doubled the number of left and right labels and helped balance all three classes.

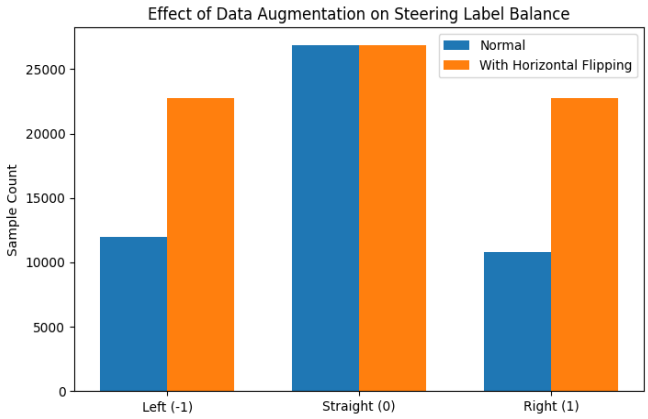


Fig. 8. The distribution of classes with and without up-sampling from horizontal flipping.

3. Using Keras class weights to assign a higher penalty (1.4) for misclassifying left labels compared to straight and right (1.0). During inference runs, the car demonstrated higher errors resulting from lack of left turn corrections.

Up-sampling through horizontal flipping yielded poor accuracy and was omitted in the final model. We hypothesized that unbalanced classes were favored because it mimicked the real-world distribution. For instance, despite the class imbalance, during training, the predicted class distribution of the test set closely mimics the ground truth. Using techniques 1 and 3, we achieve a balanced accuracy score of 0.77 on the test set, and Figure 9 shows the distributions of predictions (right) against the ground truth distribution (left).

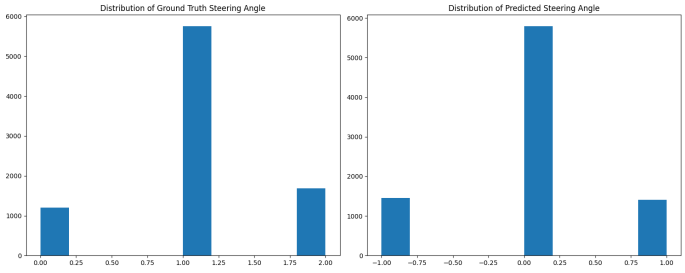


Fig. 9. Distribution of ground truth steering angles (left) and predicted steering angles (right) on the test set using techniques 1 and 3 from Section III. C.

D. Model Training and Compression

Due to memory and compute requirements of this dataset, all training was done through Google Colab using NVIDIA A100 GPUs.

We also implemented PCQAT to reduce inference time on the Raspberry Pi 5.[2] We set sparsity to 50% and used 8 centroids for weight clustering.

Stage	Test Accuracy	Model Size
Baseline (CNN)	80.45%	665.7KB
Pruning (50% Sparsity)	85.96%	413.6KB
Clustering (8 Clusters)	85.11%	83.4KB
PCQAT Model	86.08%	70.7KB

Table 1. CNN test accuracy and model size across stages of the PCQAT pipeline.

As Table 1 shows, the final PCQAT model is around 9.4 times smaller than the baseline while boasting over a 5.5% gain in accuracy.

E. Model Deployment

The trained and compressed model weights are transferred to the Raspberry Pi as a .tflite file. During an autonomously controlled run, an image is captured at every control timestep. This image is preprocessed using the same steps as outlined in Section III.A. The processed images are then passed to the model to generate a steering prediction. This prediction is used to set the steering value for that control timestep. Throttle is pulsed at the same predefined, consistent rate as during data collection. This control loop runs at 57hz, at the same frequency as during data collection. Our model is lightweight enough that its inference time does not slow down the control loop; the bottleneck remains the camera's image capture rate.

F. Expert Intervention Learning

All original training data described in Section III are collected under expert driving control. The expert is consistently able to follow the tape line closely, resulting in a dataset of state-action labels in "good" states. Therefore, the policy we learn has been trained to make predictions in these "good" states. However, as the policy makes small steering prediction errors at runtime, the car will deviate from the tape line and visit states that it did not often see during training. As a result, the policy makes worse predictions on this out of distribution data. The technique we chose to address this challenge is EIL [1]. EIL approximates the no-regret guarantee of DAgger [4]. Unlike DAgger, which requires the human to label states visited by the car, EIL allows for much more efficient collection of high-quality additional training examples of states visited by the car.

To implement EIL, during autonomous driving, the human operator retains the ability to override the autonomous policy and provide "correction" inputs when the agent makes mistakes. The expert will allow the policy to retain control of steering as long as the car remains in a "good enough" state. In our case, the track is tight, with walls on either side for most of the way around. Therefore, a "good enough" state is one which a collision is not directly impending. As soon as the car reaches a state from which a collision is impending, the expert will intervene, providing steering inputs to guide the car back to the center of the track. These corrections are logged at roughly 29hz in the same format as training data, described in Section III. Once the expert has guided the car back to the center of the track, control is passed back to the learned policy.

The expert intervention data is concatenated with the original training dataset and used to retrain a new model. Each retraining cycle adds roughly 10% more samples to the training dataset. The model training techniques remain the same as in Section IV.

This forms an iterative process. A policy from iteration i is used to collect expert intervention data. This data is used to train the model for policy $(i + 1)$. We test the real-world driving performance across various iterations of EIL and discuss the results of this experiment in Section IV.B.

G. Regression vs Classification

Initially, we framed the steering task as a regression problem with mean squared error as a loss function. However, in practice, this approach was unable to produce rational steering direction predictions. We hypothesized that the regression model struggled to produce a scalar value with the noise in our data collection. Without distinct decision boundaries like in a classification task, the model failed to converge on correct direction predictions.

We pivoted to a classification model with softmax activation, and it performed significantly better. It was more robust to input noise and reduced inverse steering direction predictions. Therefore, all other sections of this study focus on the classification model.

H. Multi-Input Model

To improve the model's context awareness, we experimented with a multi-modal architecture. It fused image input with the throttle value (on/off). Our hypothesis was that adding an extra feature could help the car navigate sharp corners or learn additional context related to the car's speed.

The image was passed through the CNN in Table 2, and the throttle value was passed through a small fully connected network with ReLU activation. These two outputs were concatenated before entering multiple dense layers for a final output.

Contrary to our hypothesis, adding this extra feature made no difference in the car's ability to navigate and failed to yield any gain in accuracy. We think since the throttle was pulsed in a fixed, open-loop pattern instead of being continuously controlled by a driver, it provided little to no significance to our final steering angle. For our final model, we removed the throttle input to reduce model size and inference latency.

We also experimented with adding a history of previous steering commands. We use a vector of 20 steering values representing commands from the last 2 seconds, taken at 0.1 second intervals. This vector is concatenated with the output from the final CNN layer. We hypothesized that this information

[2] Code related to PCQAT was directly used from Trevor's Assignment 3 q5.ipynb and TensorFlow's PCQAT example.

could help the model learn patterns related to the car's trajectory relative to the line. However, in practice, training accuracy became over 97% while validation and test accuracy plummeted into the teens. The reason for this may be that previous steering commands for a given training image are the labels for other training images in the dataset. This results in a form of data leakage where the model memorizes metadata to make accurate predictions. This leakage could be prevented by omitting training images whose labels appear in the metadata of other images, but it would reduce our sample size by 20x. We chose to omit previous steering commands in our final model, but it remains a promising area for future work and data collection efforts.

V. Experiments

A. Model

We designed a custom CNN architecture inspired by the NVIDIA PilotNet model [2]. While PilotNet was originally designed for regression, we used ours for classification. Another change is using Batch Normalization which allowed for a higher initial learning rate. One last addition was dropout and L2 Kernel Regularization to reduce overfitting.

Layer	Type	Configuration	Output Shape	Parameters
Input	Image	60 × 80 × 3 (YUV)	(60,80,3)	**0**
1	Conv2D + BN + ReLU	Filters: 24, Kernel: 5×5, Stride: 1	(60,80,24)	**1872**
2	Conv2D + BN + ReLU	Filters: 36, Kernel: 5×5, Stride: 2	(30,40,36)	**21708**
3	Conv2D + BN + ReLU	Filters: 48, Kernel: 3×3, Stride: 2	(15,20,48)	**15696**
4	Conv2D + BN + ReLU	Filters: 48, Kernel: 7×7, Stride: 7	(3,3,48)	**113040**
5	Flatten + Dropout	Rate: 0.2	-432	**0**
6	Dense + BN + ReLU	Neurons: 64, Dropout: 0.2	-64	**27904**
7	Dense + BN + ReLU	Neurons: 24, Dropout: 0.2	-24	**1632**
Output	Dense (Softmax)	Neurons: 3 (Left, Straight, Right)	-3	**75**
		Total Parameters:		**181927**

Table 2. CNN architecture.

For callbacks, we use early stopping, model checkpointing (and restoring best weights), and an exponentially decaying learning rate. The model is trained for a maximum of 30 epochs with sparse categorical cross-entropy loss and Adam optimizer. Training and validation curves can be seen in Figure 7.

B. Driving Performance across Iterations

To systematically evaluate the impact of increasing EIL iterations, we empirically test the performance of the model learned at each iteration on our track. We utilize two metrics to do this. The first metric is the average distance traveled autonomously. To compute this metric, the car starts at 5 equally spaced positions on the track and is allowed to navigate fully autonomously until a collision. We then measure the distance along the track traveled at each starting position and average them. The next metric captures the percentage of driving done autonomously by the policy over one lap. The car starts from a fixed point on the track and allowed to navigate autonomously before reaching an irrecoverable state (one where a collision is imminent). At this point, the expert intervenes and guides the car back to the track before immediately handing control back to the policy. We then calculate the percentage of the lap that was completed under policy control. To offer a baseline, we compute the same metrics using a policy that chooses random steering directions at each control timestep. The results of this study can be seen in Table 3.

Iteration	Avg Distance Traveled (inches)	Percent Autonomous
0	67.6	77.8%
1	198.3	83.2%
2	115.4	91.3%
3	197.8	82.9%
4	72.0	97.1%
5	93.6	80.8%
6	58.7	82.1%
7	127.5	84.3%
8	116.4	72.7%
Random	44.0	58.3%

Table 3. Comparison of Driving Performance Across Iterations of EIL

All the behavior cloning policies outperform the random policy in both metrics by a substantial margin. The best behavior cloning policy achieved an average distance traveled that was 4.5 times further than the random policy.

In both metrics, one iteration of EIL provided substantial performance improvements over the base model. This was apparent while watching the car. When centered over the line, the base model would do a decent job following the track, but it would eventually drift to a difficult state and fail to recover. After providing a few laps of corrections from these failure points, the resulting model had a noticeable ability to recover specific failure points, even when the car had drifted far from the track.

Although the metrics and empirical observation suggested improvement in driving performance after one EIL iteration, adding additional iterations did not always offer improvements. The average distance traveled metric is highest after only one EIL iteration. While the average percent autonomous consistently improves through the first two iterations, iteration 8 sees the lowest value. The lack of monotonic improvement could be due to the data distribution shift that occurs when adding new observations to the training data from expert corrections. These new observations de-prioritize the original clean expert driving data, which can eventually cause the model performance on the real-world driving task to regress. Empirically, a later iteration of the policy would be able to succeed in sections of the track that the preceding iteration struggled in, but it would simultaneously introduce new failure points in the track that the preceding iteration had been able to succeed in.

Some of the variance in these metrics could be attributed to the metrics themselves. For instance, the average percentage autonomous is dependent on how close to failure the expert waits to override control and how soon after recovery they hand it back to the model. There was an effort to keep this consistent; however, human variance may introduce some skew in this metric. Averaging the percentage autonomous over a larger number of laps and even over multiple different drivers could help smooth out these fluctuations.

C. Saliency Maps and Failure Analysis

To inspect the black box decision making of our model, we used Gradient-weight Class Activation Mapping (Grad-CAM) [8]. Qualitative analysis of the heatmaps revealed that the model often but not always had a heatmap that aligned with the blue tape.

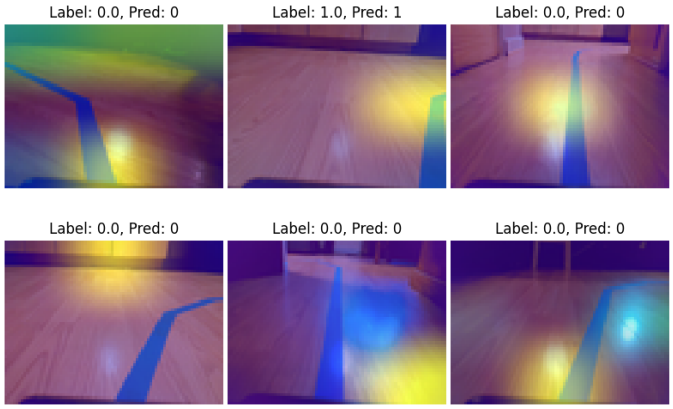


Fig. 10. Grad-CAM class activation maps showing saliency maps on correctly predicted images.[3]

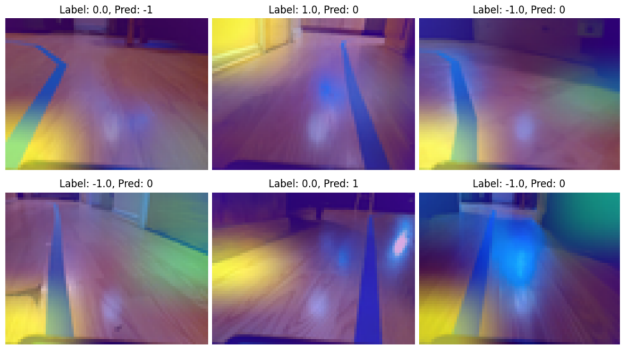


Fig. 11. Grad-CAM class activation maps showing saliency maps on mispredicted images.

Figure 10 shows activation maps on 6 random images from the training set which were predicted correctly, and Figure 11 does the same for images which were incorrectly predicted. Yellow areas represent parts of the image which contribute most strongly to the model's class prediction. The yellow regions often intersect with areas directly in front of the car. This may suggest that the model is identifying the position of the line in the frame. Because the car moves relatively slowly, it would make sense that the position of the line directly in front of the car is most important. However, this is not always the case. In some images, the region with highest saliency intersects with the position of the line further ahead. In others, this region is unrelated to the line, appearing on cabinets or other furniture. The variance in what features the model finds important suggest that behavior cloning techniques could offer greater flexibility than more hard coded policies. For instance, the model was able to learn that certain cabinets signify the need to turn left. This allows the model to recover from difficult states in this specific track formation, even when the line is not in frame. However, learning features also means that the agent's behaviors are harder to guarantee. While the position of the line in the frame is often enough information to reliably follow a track [7], the model sometimes focuses on other features such as furniture. This leads to more unreliable behavior, especially when conditions change from the training data. Perhaps by purposely altering more exterior conditions of the track during training, the model would be more forced to rely on the unchanging important feature of line position.

We also attempt to compare Figures 10 and 11 to identify why the model makes mistakes. For instance, Figure 10 shows saliency regions at or near the line for all but the bottom left image. The bottom left image highlights a cabinet that would be in view when the line disappears during a sharp left turn. These appear plausible. In contrast, while many images in Figure 11 also appear plausible, the center two images highlight seemingly more irrelevant aspects of the image. One might be tempted to infer that the failed feature extraction in these images is responsible for the misprediction. This may hold some truth, perhaps our training samples offered a misleading correlation between the specific location of these irrelevant features and steering commands.

The saliency maps do highlight interesting insights into the feature extraction process; it was rewarding to see that the line position often appeared with high saliency values, and it suggests that our models may have overfit to confounders in our training data. However, these maps represent gradient values and do not directly show causal importance or model reasoning. Therefore, we feel it is unreasonable to make any definitive conclusions on *why* the model made the predictions it made from these maps alone.

D. Failure Examples Analysis

Figure 11 can give additional insights into when the model mis-predicted, without needing to consider the saliency heatmaps. The images show ground truth labels and the model's predictions above them. A label of -1 is left, 0 is straight, and 1 is right. Manually examining this sample of mispredictions shows that the model's choices are often reasonable even when they contradict the expert's steering commands. For instance,

[3] Code related to Grad-CAM was directly used from "Emerging Directions Part 2" slides.

the top left image is labeled as straight, but the model predicted left. The line in this image is to the left of the frame. This highlights a challenge in our problem. The labels come from a human's in the moment decisions. These decisions include human bias and potential mistakes — incredibly difficult phenomenon to model. We considered the possibility of collecting training data under more deterministic demonstrations by using a PID controller [7]. However, we feel that the strengths of behavior cloning come from the potential to apply it in scenarios where humans can complete tasks that classical control paradigms cannot. Research into behavior cloning should therefore focus on learning from humans despite the added variance. The general success of our model suggests that with enough examples, the average expert driving behaviors can still be learned.

While some mispredictions could be attributed to the unpredictable human decision-making, another source of error could be the discrepancy between state information available to the expert and available to the model. The human expert observes the car in real-time from a third-person perspective. The expert is therefore able to understand the current velocity of the car and predict future velocity under the current trajectory. A single image does not capture this information. The line could be to the left of the image, but if the car is already heading left, an appropriate steering direction may be straight or even right. This is the purpose of the derivative term in classical PID controllers. A potential solution to this challenge is including additional information in the inputs to the model. Section IV.H discusses experiments and challenges related to including a history of directly preceding steering commands. Accelerometer readings are another option to capture velocity of the vehicle but require additional hardware. Finally, including a time series of images could allow the model to learn trajectories, but it would increase the computational requirements of the embedded platform.

Our analysis suggests that two additional potential failure points of the model predictions emerge from incomplete information and human variance. In this work, we chose to tolerate some percentage of mispredictions in favor of reducing complexity of our model and hardware.

VI. Conclusion

A. Key Results

In this project we implement an end-to-end behavior cloning approach with EIL on a scale car in the real world. We create a data collection pipeline to allow for teleoperation and logging. We collect various datasets and train numerous models, experimenting with different hyperparameters, architectures, and paradigms. We detail the choices which provide the best performance and offer discussions into choices that did not work.

A key contribution of our work is a study of the empirical impact of increasing EIL iterations to combat the distribution shift problem in behavior cloning. This includes the creation of an EIL script to interface with our hardware for correction collection along with a contrastive analysis of the real-world performance over 8 EIL iterations. We find that increasing EIL iterations past one in our framework does not yield consistent performance improvements.

Our best design choices result in a fully learned policy that displays intelligent track following abilities, including completing a fully autonomous lap on a tight track. This control strategy is gained without encoding any information about the problem itself, and it can recover from states where the track is not visible.

Finally, we examine failure cases and saliency maps to gain insight into why the model makes inaccurate predictions. We identify overfitting to irrelevant confounders, incomplete state information, and human variance as key challenges that inhibit our policy from indefinite collision-free driving.

B. Lessons Learned

A key takeaway from this project is how fragile relying on a single sensor is for autonomous control. Our vision only CNN could achieve one complete lap; however, it was susceptible to lighting shifts and background noise (even with extensive data augmentation). It highlights the importance of sensor fusion in real-world autonomous systems.

Another takeaway is how data quality outweighs model complexity. Performance gains came from more training data, effective augmentation, and an EIL iteration. Furthermore, simplifying the model to a classification problem instead of a regression problem produced a more robust control strategy.

Lastly, we observed the risks and tradeoffs of class imbalance in behavioral cloning. Most driving data has a straight label. Therefore, the model tended to converge on picking this class most often. Yet, we noticed that a class imbalance may be preferred depending on the problem. In our case, a dataset with majority straight labels proved to be the most effective.

C. Future Extensions

Future work involves incorporating velocity into predictions. The throttle is continuously pulsed, which omits velocity from model control. However, a proper multi-output regression model could allow the model to learn to slow down before sharp corners, or to increase speed during straightaways.

We also are interested in the incorporation of additional metadata inputs. As discussed in Section IV.D, including accelerometer data, previous steering commands, or a time-series of images are promising solutions to a key inhibitor of incomplete state that our model dealt with. Using a Recurrent Neural Network or Long Short-Term Memory network instead of a CNN could be an effective choice given the time-series aspect of our proposed data. It could be used to prevent a "jittery" behavior when the car oscillates between classes.

REFERENCES

[1] J. Spencer *et al.*, "Expert intervention learning," *Autonomous Robots*, vol. 46, pp. 99–113, 2022.

[2] M. Bojarski *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[3] F. Torabi, G. Warnell, and P. Stone, "Behavioral cloning from observation," in *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018, pp. 2635-2641.

[4] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627-635.

[5] A. Haji, P. Shah, and S. Bijoor, "Self driving RC car using behavioral cloning," *International Research Journal of Engineering and Technology (IRJET)*, vol. 6, no. 10, pp. 1-6, Oct. 2019.

[6] S. I. Morga-Bonilla *et al.*, "Behavioral cloning strategies in steering angle prediction: Applications in mobile robotics and autonomous driving," *World Electric Vehicle Journal*, vol. 15, no. 11, p. 486, 2024.

[7] T. Chartier, "LAYMO," GitHub repository, 2025. [Online]. Available: https://github.com/TrevorChartier/LAYMO

[8] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618-626.